

ECOM EXPERTS

Ecom PriceSearch

Manual de usuario e instalación

ecom_pricesearch 1.0.0 · 08/09/2026

Contenido

1. Qué hace y para quién

2. Compatibilidad

3. Instalación

4. Configuración

Conexión API

Sincronización

Repricing

Cron

5. Cómo se usa, día a día

5.1 Primera sincronización del catálogo

5.2 Mantenerlo al día: el cron

5.3 El Panel

5.4 Repricing

5.5 Coincidencias

6. Qué ve el cliente en la tienda

7. Preguntas frecuentes y problemas

8. Ficha técnica

Versión 1.0.0 · Ecom Experts · Licencia AFL-3.0

Este manual cubre **la parte que se instala en la tienda**: qué hace el módulo dentro de PrestaShop, cómo se configura, qué envía al servicio PriceSearch y qué ve el comerciante. Todo lo que pasa en el servidor (alta de cuenta, claves de API, competidores, rastreo, facturación y despliegue) está en el manual del proyecto, en la carpeta hermana `manuales/pricesearch/` (`pricesearch-manual.pdf` , `PRESTASHOP.md` , `ARCHITECTURE.md` , `DEPLOY.md` , `BILLING.md`).

1. Qué hace y para quién

Ecom PriceSearch es el conector entre una tienda PrestaShop y el servicio de comparación de precios **PriceSearch** (<https://pricesearch.gmartos.es>). El servicio es quien rastrea las webs de la competencia; el módulo es quien le cuenta qué productos vendes y quien enseña, dentro del back-office, a qué precio los tiene cada competidor.

Sirve a un comerciante que vende productos identificables (con EAN, referencia o nombre reconocible) y quiere saber, sin salir de PrestaShop, si está por encima o por debajo del mercado. El **Panel** muestra, producto a producto, tu precio junto al mínimo, medio y máximo de la competencia, cuántos competidores lo venden y un semáforo de color.

Además permite revisar las **coincidencias** que el servicio propone (este producto tuyo es este otro artículo en tal web) y aprobarlas o rechazarlas con un clic, y aplicar **repricing**: bajar el precio un céntimo por debajo del competidor más barato, siempre respetando un margen mínimo sobre el precio de coste, un suelo y un techo. El repricing puede funcionar en **simulación**, que solo anota lo que haría, o en **automático**, que cambia de verdad los precios de la tienda.

El módulo no toca nada del escaparate: el cliente final no ve ni un píxel distinto. Todo ocurre en el back-office y en un cron.

2. Compatibilidad

PrestaShop	Declara <code>1.7.8.0</code> a <code>8.99.99</code> . Probado e instalado en 8.2.7 . En PrestaShop 9 el instalador lo rechazará por esa declaración, aunque el código no usa nada exclusivo de 8.
PHP	7.4 y 8.x. Necesita la extensión cURL (todas las llamadas al servicio van por cURL con verificación SSL).
Multitienda	La configuración se guarda con <code>Configuration::updateValue</code> (por tienda/contexto). La instalación fuerza el contexto a todas las tiendas. El repricing de combinaciones crea precios específicos en la tienda del contexto actual.
Idiomas	Los textos del módulo están escritos directamente en castellano (sin tildes en pantalla). Hay <code>translations/es.php</code> y <code>en.php</code> legados, pero el módulo declara el sistema nuevo de traducciones y las plantillas usan <code>mod=</code> , así que en la práctica los textos salen tal cual están en el código.
Dependencias	Ninguna librería en el módulo. El gráfico del Panel carga Chart.js 4.4.1 desde <code>cdn.jsdelivr.net</code> : sin salida a internet desde el navegador del empleado no se pinta el gráfico (la tabla sí).
Servicio	Necesita una cuenta activa en PriceSearch con API Key y API Secret . Sin ellas el módulo instala y se abre, pero «Sincronizar ahora» y «Probar conexión» devuelven <i>Configuración API incompleta</i> . Durante la elaboración de este manual el servicio <code>pricesearch.gmartos.es</code> no estaba accesible , así que las pantallas con datos se han generado con registros de ejemplo en las tablas del módulo.

3. Instalación

- Módulos → Gestor de módulos → **Subir un módulo** → `ecom_pricesearch.zip`.
- Al instalar, el módulo: - Registra cuatro hooks: `actionProductUpdate`, `actionProductSave`, `displayBackOfficeHeader` y `actionAdminControllerSetMedia`. - Crea la pestaña **Ecom PriceSearch** dentro de *Módulos* (clase `AdminEcomPricesearch`, colgada de `AdminParentModulesSf`). - Crea tres tablas: `ps_ecom_pricesearch_price`, `ps_ecom_pricesearch_match` y `ps_ecom_pricesearch_history` (ver ficha técnica). - Escribe la configuración por defecto: endpoint `https://pricesearch.gmartos.es`, frecuencia diaria, precios con impuestos, repricing desactivado en modo simulación, margen mínimo 10 %, redondeo a .99 y un **token de cron** aleatorio de 32 caracteres.
- El botón **Configurar** del gestor de módulos no abre un formulario: redirige a la pestaña *Módulos* → *Ecom PriceSearch*. Antes de redirigir ejecuta `addnewfeatures()`, que vuelve a registrar hooks o pestaña si faltan y regenera el token y el endpoint si se hubieran borrado.

Al desinstalar se borran las tres tablas (con todo su histórico), la pestaña y las doce claves de configuración. En el servicio PriceSearch no se borra nada.

Instalado en la tienda local 8.2.7 (`instalar_en_locales.py`): 3 tablas, 4 hooks, 1 pestaña, `FALLAN: 0`.

4. Configuración

Todo está en **Módulos** → **Ecom PriceSearch**, una pantalla con tres pestañas: *Panel*, *Coincidencias* y *Ajustes*. Lo primero es rellenar *Ajustes*.

The screenshot shows the PrestaShop 8.2.7 interface with the 'Ecom PriceSearch' module configuration page. The left sidebar contains navigation menus for 'VENDER' (Pedidos, Catalogo, Clientes, Servicio al Cliente, Estadísticas, Puntos de fidelidad, Presupuestos) and 'PERSONALIZAR' (Modulos, Creative Elements, Diseño, Transporte, Pago, Internacional). The 'Modulos' menu is expanded, showing 'Administrador de modulos', 'PTS - One Page Checkout', 'PrestaShop', 'Abandcart AI', and 'Ecom PriceSearch'. The main content area is titled 'Ecom PriceSearch' and has three tabs: 'PANEL', 'COINCIDENCIAS', and 'AJUSTES'. The 'AJUSTES' tab is active, showing the following configuration sections:

- Conexion API**
 - Endpoint del SaaS:
 - API Key:
 - API Secret:
- Sincronizacion**
 - Frecuencia:
 - Precios con impuestos:
- Repricing**
 - Repricing activado:
 - Modo:
 - Margen minimo (%):
 - Suelo de precio (0 = sin suelo):
 - Techo de precio (0 = sin techo):
 - Redondeo a .99:
- Cron**
 - URL del cron:

At the bottom of the configuration area, there are two buttons: 'GUARDAR AJUSTES' (Save Settings) and 'Probar conexion' (Test Connection).

Conexión API

Campo	Efecto
Endpoint del SaaS	URL base del servicio, sin barra final. Solo admite <code>http://</code> o <code>https://</code> ; cualquier otra cosa se rechaza con un aviso y no se guarda. Vacío desactiva las llamadas.
API Key	La clave que te da el panel de PriceSearch (formato <code>prefijo.clave</code>). Viaja en la cabecera <code>X-API-Key</code> .
API Secret	El secreto HMAC. No se envía nunca : se usa para firmar cada petición (<code>HMAC-SHA256</code> de <code>timestamp.cuerpo</code>) en la cabecera <code>X-Signature</code> .

Sincronización

Campo	Efecto
Frecuencia (cada hora / diaria / semanal)	Se guarda como preferencia, pero el módulo no programa nada por sí mismo : la cadencia real la marca la tarea cron que configures en el servidor (apartado 5). Ponla acorde.
Precios con impuestos	Decide qué precio se manda como <code>price</code> principal al servicio (con o sin IVA). En cualquier caso se envían siempre los dos, <code>price_tax_excl</code> y <code>price_tax_incl</code> .

Repricing

Campo	Efecto
Repricing activado	Con <i>No</i> , el botón «Ejecutar repricing» y el cron contestan <i>Repricing desactivado</i> y no tocan nada.
Modo	<i>Simulación (dry-run)</i> : calcula el precio nuevo y lo apunta en el histórico, sin cambiar nada. <i>Automático</i> : cambia el precio de verdad. Cualquier otro valor se guarda como simulación.
Margen mínimo (%)	Porcentaje sobre el precio de coste (<code>wholesale_price</code>) por debajo del cual nunca se baja. Si el producto no tiene precio de coste, este freno no actúa.
Suelo de precio	Precio mínimo absoluto. <code>0</code> = sin suelo.
Techo de precio	Precio máximo absoluto. <code>0</code> = sin techo.
Redondeo a .99	Tras aplicar los frenos, deja el precio en <code>X,99</code> (parte entera + 0,99).

Cron

Muestra la **URL del cron** con el token ya incluido. Es de solo lectura; cópiala tal cual al programador de tareas del servidor.

Botones: **Guardar ajustes** guarda todo lo anterior de una vez; **Probar conexión** hace un `POST /api/v1/webhooks/test` firmado y responde *Conexión correcta con el SaaS PriceSearch* o *Fallo de conexión: ...* con el motivo (`Configuración API incompleta` , `cURL: ...` o `HTTP nnn`).

5. Cómo se usa, día a día

5.1 Primera sincronización del catálogo

En *Panel*, pulsa **Sincronizar ahora**. El módulo recorre **todos los productos activos**, serializa cada uno (y cada combinación por separado) y los manda a `POST /api/v1/catalog/sync` en lotes de 200. Al terminar dice *Sincronizados N registros en M lotes*. Si un lote falla, se detiene ahí y muestra el error.

Por cada producto o combinación se envía:

Campo	De dónde sale
<code>ext_id</code>	<code>id_product</code> , o <code>id_product-id_product_attribute</code> si es una combinación
<code>id_product</code> , <code>id_product_attribute</code>	identificadores de PrestaShop
<code>sku</code> , <code>ean13</code> , <code>upc</code> , <code>mpn</code>	de la combinación; si están vacíos, del producto
<code>name</code> , <code>brand</code>	nombre en el idioma del empleado y nombre del fabricante
<code>price</code>	con o sin IVA según <i>Precios con impuestos</i>
<code>price_tax_excl</code> , <code>price_tax_incl</code>	siempre los dos
<code>currency</code>	ISO de la moneda del contexto (por defecto <code>EUR</code>)

No se envían descripciones, imágenes, stock ni datos de clientes.

5.2 Mantenerlo al día: el cron

Cada vez que se guarda un producto (hooks `actionProductUpdate` / `actionProductSave`), el módulo lo marca como **pendiente** (`dirty = 1`) en su tabla de precios. No llama al servicio en ese momento: lo hace el cron. Programa en el servidor la URL de *Ajustes* → *Cron*:

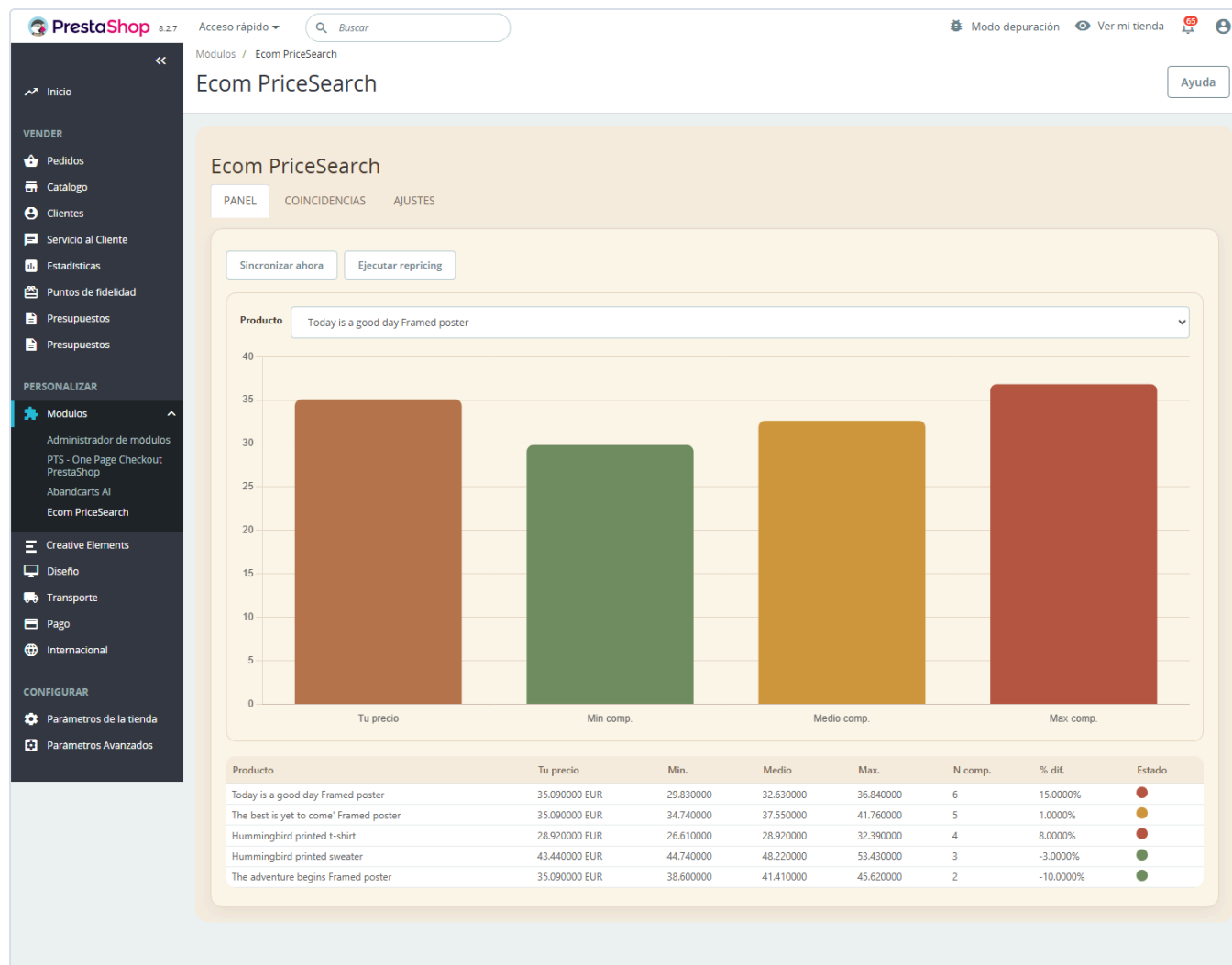
```
0 3 * * * curl -s "https://TU-TIENDA/module/ecom_pricesearch/cron?token=XXXX" > /dev/null
```

Cada ejecución hace tres cosas, en este orden, y devuelve un JSON con el resultado:

1. **sync** : envía al servicio solo los productos pendientes y les quita la marca.
2. **prices** : para cada fila de la tabla de precios pide `GET /api/v1/prices?product_ext_id=...` y guarda mínimo, medio, máximo, número de competidores y % de diferencia.
3. **repricing** : si está activado, ejecuta el repricing (apartado 5.4).

Sin token o con token equivocado responde `403` y `{"ok":false,"error":"Invalid token"}`. Comprobado en la tienda local: con token bueno devuelve `{"ok":true,"sync":{"...},"prices":{"...},"repricing":{"...,"error":"Repricing desactivado"}}}`.

5.3 El Panel



La tabla lista los productos de la tabla local ordenados por número de competidores y diferencia. Columnas: tu precio, mínimo, medio y máximo de la competencia, número de competidores, % de diferencia y el semáforo **Estado**:

- **Verde**: tu precio es igual o menor que el mínimo de la competencia.
- **Ámbar**: estás hasta un 5 % por encima del mínimo, o no hay datos todavía.
- **Rojo**: más de un 5 % por encima del mínimo.

Arriba, el desplegable **Producto** cambia el gráfico de barras (tu precio, mínimo, medio y máximo del producto elegido). El gráfico necesita Chart.js desde internet; si no carga, la tabla sigue funcionando.

Mientras no haya filas, la tabla dice *Aún no hay datos de precios. Sincroniza el catálogo.*

5.4 Repricing

Con **Ejecutar repricing** (o desde el cron), para cada producto con precio mínimo de competencia conocido:

1. Objetivo = mínimo del competidor – 0,01.
2. Se sube hasta $\text{coste} \times (1 + \text{margen mínimo})$ si quedaba por debajo.
3. Se aplica el suelo y el techo.
4. Si *Redondeo a .99* está activo, se deja en $X,99$.
5. Si el resultado difiere menos de un céntimo del precio actual (sin IVA), no se hace nada.

En **simulación** solo se escribe una línea en `ps_ecom_pricesearch_history` con precio antiguo, nuevo y modo `dry-run`. En **automático**, para un producto simple se cambia `product.price` y se guarda el producto; para una combinación se crea (o actualiza) un **precio específico** fijo de esa combinación, desde 1 unidad, sin fechas ni condiciones, en la tienda del contexto. El mensaje final es *Repricing (modo): N procesados, M con cambio*.

Recomendación: déjalo en simulación unos días y revisa el histórico antes de pasar a automático. El módulo no tiene pantalla para el histórico: se consulta en la tabla `ps_ecom_pricesearch_history`.

5.5 Coincidencias

The screenshot shows the PrestaShop 8.2.7 interface for the Ecom PriceSearch module. The 'COINCIDENCIAS' tab is active, displaying a table of product matches. The table has the following data:

Producto	Competidor	Título remoto	Precio	Score	Metodo	Acciones
Hummingbird printed t-shirt	tiendaejemplo.es	Camiseta estampada colibri talla S	21.900000	0.9312	ean	<button>Aprobar</button> <button>Rechazar</button>
Hummingbird printed sweater	competidor-online.com	Taza de cerámica colibri 300 ml	12.500000	0.8720	title	<button>Aprobar</button> <button>Rechazar</button>
The best is yet to come' Framed poster	outletmoda.es	Cojín colibri 40x40	19.990000	0.7415	title	<button>Aprobar</button> <button>Rechazar</button>

Lista las coincidencias **pendientes** de la tabla local: producto tuyo, competidor, título del artículo remoto (enlace, solo si es `http/https`), precio, puntuación (`score`) y método de emparejamiento (`ean` , `title` ...).

Aprobar o **Rechazar** cambia el estado en la tabla local y avisa al servicio (`POST /api/v1/matches/{id}/approve|reject`). Si el servicio no contesta, la fila se actualiza igual localmente y se avisa: *Actualizada localmente; el SaaS devolvió:*

Ojo: en esta versión **el módulo no descarga las coincidencias** del servicio (la llamada `GET /api/v1/matches` está preparada en la clase de API pero no la usa ni el cron ni el panel). La tabla se rellena desde el lado del servicio o a mano; consúltalo en el manual del proyecto.

6. Qué ve el cliente en la tienda

Nada. El módulo no registra ningún hook de front ni modifica plantillas. Lo único visible desde fuera es la URL del cron, que exige el token. El efecto indirecto es el precio: si el repricing está en automático, los precios del catálogo cambian y el cliente los ve como cualquier otro cambio de precio.

7. Preguntas frecuentes y problemas

«Configuración API incompleta» al sincronizar o probar. Falta endpoint, API Key o API Secret. Los tres son obligatorios.

«Fallo de conexión: cURL: ...». El servidor de la tienda no llega al servicio (DNS, firewall, puerto 443, certificado). Durante la redacción de este manual el servicio no estaba en línea y este era el resultado.

«Fallo de conexión: HTTP 401/403». Clave o secreto incorrectos, o reloj del servidor desfasado: la firma lleva el `timestamp` y el servicio la rechaza si difiere demasiado.

El Panel sigue vacío después de «Sincronizar ahora». Es normal: ese botón solo *envía* el catálogo. Las filas del Panel aparecen cuando (a) se guarda un producto, que lo inscribe en la tabla local, y (b) el cron descarga sus precios de competencia. Programa el cron y edita/guarda los productos que quieras vigilar, o espera a la primera pasada.

«Tu precio» sale a 0 y el semáforo siempre en ámbar. En esta versión ninguna rutina del módulo escribe `own_price` en la tabla local: se crea a 0 al marcar el producto y solo lo rellena quien alimente la tabla desde fuera. El repricing no lo usa (lee el precio real del producto), pero el semáforo y el gráfico sí. Es una limitación conocida de la 1.0.0.

El gráfico no aparece. El navegador del empleado no ha podido cargar Chart.js desde `cdn.jsdelivr.net`. La tabla no depende de él.

La frecuencia que elijo no cambia nada. Es solo informativa; la cadencia la marca el cron del servidor.

He cambiado el modo a automático y no cambia precios. Repasa: *Repricing activado* = Sí; hay filas con precio mínimo de competencia; el precio calculado difiere al menos un céntimo del actual; el margen mínimo, suelo o techo no lo están devolviendo al precio de siempre. Cada intento con cambio deja rastro en `ps_ecom_pricesearch_history`.

Repricing y combinaciones en multitienda. El precio específico se crea en la tienda del contexto en que corre (la del empleado o la del cron). Si tienes varias tiendas, lanza el cron con la URL de cada una.

Textos sin tildes en pantalla. Los literales del módulo están escritos sin acentos en el código y las plantillas usan el sistema de traducción legado (`mod=`), que no lee los `.xlf`. Es cosmético.

8. Ficha técnica

Ficheros

```

ecom_pricesearch.php           clase Ecom_pricesearch (install, hooks,
addnewfeatures)
classes/EcomPricesearchApi.php cliente cURL firmado (HMAC-SHA256)
classes/EcomPricesearchSync.php serialización y envío del catálogo por lotes de 200
classes/EcomPricesearchRepricer.php reglas de repricing, precios específicos, histórico
controllers/admin/AdminEcomPricesearchController.php pestañas Panel / Coincidencias / Ajustes
controllers/front/cron.php      endpoint del cron (token obligatorio)
views/templates/admin/dashboard.tpl única plantilla
views/js/dashboard.js, views/css/dashboard.css
sql/install.php, sql/uninstall.php
translations/es.php, translations/en.php (sistema legado, no se consumen)

```

Tablas

Tabla	Contenido
ps_ecom_pricesearch_price	Una fila por producto/combinación: ext_id, own_price, min_price, avg_price, max_price, competitors_count, diff_pct, currency, dirty. Única por (id_product, id_product_attribute).
ps_ecom_pricesearch_match	Coincidencias: remote_match_id, competidor (competitor_name, competitor_url, competitor_title, competitor_price), status (pending / approved / rejected), score, method.
ps_ecom_pricesearch_history	Histórico de repricing: action, old_price, new_price, reason, mode (auto / dry-run), date_add.

Hooks: actionProductUpdate, actionProductSave (marcan el producto como pendiente); displayBackOfficeHeader, actionAdminControllerSetMedia (cargan CSS/JS solo en el controlador del módulo).

Controladores: admin AdminEcomPricesearch (pestaña en *Módulos*; parámetro ecom_tab = dashboard | matches | settings; acciones submitEcomSettings, ecomTestConnection, ecomSyncNow, ecomRunRepricer, ecomApproveMatch, ecomRejectMatch); front cron (/module/ecom_pricesearch/cron?token=..., sin ruta amigable propia).

Claves de configuración (ECOM_PRICESEARCH_*): ENDPOINT, API_KEY, API_SECRET, FREQUENCY, TAX_INCLUDED, REPRICING_ENABLED, REPRICING_MODE, MIN_MARGIN, PRICE_FLOOR, PRICE_CEIL, ROUND_99, CRON_TOKEN.

Llamadas al servicio (todas con cabeceras X-API-Key, X-Timestamp, X-Signature; timeout 30 s, conexión 10 s, SSL verificado, solo protocolos http/https):

Método y ruta	Quién la usa
POST /api/v1/catalog/sync	Sincronizar ahora, cron (sync)
GET /api/v1/prices?product_ext_id=	cron (prices)
POST /api/v1/matches/{id}/approve · /reject	Coincidencias
POST /api/v1/webhooks/test	Probar conexión
GET /api/v1/matches , GET /api/v1/prices/history , GET /api/v1/competitors	definidas en la clase de API, sin uso en 1.0.0

Log: el módulo no escribe ficheros de log; solo PrestaShopLogger si falla addnewfeatures() . El resultado del cron es el JSON que devuelve.